Spring Boot & Backend Annotations Guide

Exhaustive Architectural Encyclopedia of All 72 Annotations Used in LibraryMS

Runtime / Framework: Spring Boot 3.5.16 | Java 21 LTS | PostgreSQL 17	**Target Stack:** Spring Web, Security, Data JPA, AOP, Bean Validation
Total Backend Annotations: 72 Unique Annotations Cataloged & Analyzed	**Organization:** Grouped into 9 Core Architectural Categories

1. Spring Core & Inversion of Control (IoC)

@SpringBootApplication

Package: org.springframework.boot.autoconfigure.SpringBootApplication

Purpose: Bootstraps the entire Spring Boot application by combining three core annotations: @Configuration, @EnableAutoConfiguration, and @ComponentScan.

How it Works: Scans for beans in the current package and subpackages, applies sensible default auto-configurations based on classpath libraries, and registers the class as a primary configuration source.

Usage in LibraryMS: Annotates LibraryManagementSystemApplication to bootstrap the backend container.

```
@SpringBootApplication
@ConfigurationPropertiesScan
public class LibraryManagementSystemApplication {
    public static void main(String[] args) {
        SpringApplication.run(LibraryManagementSystemApplication.class, args);
    }
}
```

@Component

Package: org.springframework.stereotype.Component

Purpose: Generic stereotype annotation indicating that an annotated class is an auto-detected Spring bean.

How it Works: Spring's component scanning registers an instance in the ApplicationContext container as a singleton by default.

Usage in LibraryMS: Used on cross-cutting helpers: AuditAspect, LoginAttemptLimiter, MemberSecurity, LoanSecurity, and custom security entry points.

```
@Component("memberSecurity")
public class MemberSecurity {
    public boolean isCurrentMember(Authentication auth, Long memberId) { ... }
}
```

@Service

Package: org.springframework.stereotype.Service

Purpose: Specialized stereotype of @Component designating classes that hold business logic and service-layer responsibilities.

How it Works: Behaves identically to @Component for dependency injection, but adds semantic architectural clarity and serves as an ideal pointcut target for transaction boundaries and AOP advice.

Usage in LibraryMS: Annotates all domain services: BookService, BookCopyService, LoanService, MemberService, UserService, AuthService, AuthorService, and AuditService.

```
@Service
public class BookService {
    private final BookRepository bookRepository;
    public BookService(BookRepository repo) { this.bookRepository = repo; }
}
```

@Configuration

Package: org.springframework.context.annotation.Configuration

Purpose: Declares that a class defines one or more @Bean factory methods to configure and assemble Spring beans.

How it Works: Spring processes @Configuration classes using CGLIB proxies so that direct calls to @Bean methods return container-managed singletons rather than re-instantiating new objects.

Usage in LibraryMS: Used in SecurityConfig, ClockConfig, OpenApiConfig, and AuditConfig.

```
@Configuration
public class ClockConfig {
    @Bean
    public Clock clock() { return Clock.systemUTC(); }
}
```

@Bean

Package: org.springframework.context.annotation.Bean

Purpose: Indicates that a method produces a bean to be managed by the Spring ApplicationContext container.

How it Works: Spring invokes the annotated method during application startup, captures the return value, and registers it in the IoC container under the method's name.

Usage in LibraryMS: Used to instantiate PasswordEncoder, JwtDecoder, Clock, SecurityFilterChain, and Swagger OpenAPI objects.

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

@Value

Package: org.springframework.beans.factory.annotation.Value

Purpose: Injects property values, environment variables, or SpEL expressions into bean fields or constructor parameters.

How it Works: Spring resolves the placeholder expression against the Environment configuration hierarchy with optional fallback values.

Usage in LibraryMS: Used in SecurityConfig to read CORS allowed origins from application.properties or environment variables.

```
@Value("${cors.allowed-origins:http://localhost:3000}")
private String allowedOrigins;
```

@ConfigurationProperties

Package: org.springframework.boot.context.properties.ConfigurationProperties

Purpose: Binds external configuration keys with a common hierarchical prefix (e.g. 'jwt', 'app.loans') to a strongly typed Java class.

How it Works: Spring Boot binds matching keys from application.properties or environment variables to class setters/records, supporting validation and nested properties.

Usage in LibraryMS: Used on JwtProperties (jwt.secret, expirations) and LoanProperties (app.loans duration, fine amounts).

```
@ConfigurationProperties(prefix = "jwt")
public class JwtProperties {
    private String secret;
    private long accessTokenExpirationSeconds;
}
```

@ConfigurationPropertiesScan

Package: org.springframework.boot.context.properties.ConfigurationPropertiesScan

Purpose: Automatically searches the base package for @ConfigurationProperties classes and registers them without requiring explicit @EnableConfigurationProperties.

How it Works: Runs during component scanning and discovers configuration beans across all subpackages.

Usage in LibraryMS: Placed on LibraryManagementSystemApplication so JwtProperties and LoanProperties are automatically registered.

```
@SpringBootApplication
@ConfigurationPropertiesScan
public class LibraryManagementSystemApplication { ... }
```

@PostConstruct

Package: jakarta.annotation.PostConstruct

Purpose: Marks a method to be executed once right after dependency injection is complete and the bean is fully initialized.

How it Works: Spring invokes this method after all properties are set and constructor execution finishes, but before the bean handles any external traffic.

Usage in LibraryMS: Used in JwtProperties to validate that JWT_SECRET is at least 256 bits (32 bytes) long, failing startup fast if insecure.

```
@PostConstruct
public void validate() {
    if (secret == null || secret.getBytes(StandardCharsets.UTF_8).length < 32) {
        throw new IllegalStateException("JWT secret must be at least 32 bytes");
    }
}
```

@Order

Package: org.springframework.core.annotation.Order

Purpose: Defines the precedence order of Spring components, filters, or AOP aspects.

How it Works: Lower integer values indicate higher precedence (e.g. Order(0) executes before Order(100)).

Usage in LibraryMS: Used in AuditConfig (order=0 for transaction management) and AuditAspect (order=100) so auditing runs inside active database transactions.

```
@Aspect
@Component
@Order(100)
public class AuditAspect { ... }
```

2. Spring Web & REST API Controllers

@RestController

Package: org.springframework.web.bind.annotation.RestController

Purpose: Convenience annotation combining @Controller and @ResponseBody. Marks classes as RESTful web controllers.

How it Works: Instructs Spring MVC to write method return values directly into the HTTP response body (serialized as JSON via Jackson) rather than rendering an HTML view template.

Usage in LibraryMS: Annotates all HTTP REST endpoints: BookController, LoanController, AuthController, UserController, MemberController, AuthorController, BookCopyController, and AuditController.

```
@RestController
@RequestMapping("/api/v1/books")
public class BookController { ... }
```

@RequestMapping

Package: org.springframework.web.bind.annotation.RequestMapping

Purpose: Maps web requests to specific controller classes or handler methods with URL paths and HTTP verbs.

How it Works: Registers routes with RequestMappingHandlerMapping. When used at class level, serves as a common URL prefix.

Usage in LibraryMS: Used as the root route prefix across all controllers (e.g. @RequestMapping("/api/v1/loans")).

```
@RestController
@RequestMapping("/api/v1/loans")
public class LoanController { ... }
```

@GetMapping

Package: org.springframework.web.bind.annotation.GetMapping

Purpose: Composed shortcut annotation that maps HTTP GET requests onto specific controller handler methods.

How it Works: Shortcut for @RequestMapping(method = RequestMethod.GET). Reads safe, idempotent data.

Usage in LibraryMS: Used on list and detail read operations across BookController, MemberController, LoanController, UserController, and AuditController.

```
@GetMapping("/{id}")
public ResponseEntity getBookById(@PathVariable Long id) {
    return ResponseEntity.ok(bookService.getBookById(id));
}
```

@PostMapping

Package: org.springframework.web.bind.annotation.PostMapping

Purpose: Composed shortcut annotation that maps HTTP POST requests onto specific controller handler methods.

How it Works: Shortcut for @RequestMapping(method = RequestMethod.POST). Creates new resources or triggers stateful commands.

Usage in LibraryMS: Used on resource creation: creating books, registering users, borrowing books, returning loans, and logging in.

```
@PostMapping
public ResponseEntity createLoan(
        @Valid @RequestBody CreateLoanRequest request,
        @AuthenticationPrincipal CurrentUserPrincipal principal) { ... }
```

@PutMapping

Package: org.springframework.web.bind.annotation.PutMapping

Purpose: Composed shortcut annotation that maps HTTP PUT requests onto specific controller handler methods.

How it Works: Shortcut for @RequestMapping(method = RequestMethod.PUT). Performs full idempotent resource replacements.

Usage in LibraryMS: Used on update operations: updating book metadata, modifying member profile data, and updating authors.

```
@PutMapping("/{id}")
public ResponseEntity updateBook(
        @PathVariable Long id,
        @Valid @RequestBody UpdateBookRequest request) { ... }
```

@PatchMapping

Package: org.springframework.web.bind.annotation.PatchMapping

Purpose: Composed shortcut annotation that maps HTTP PATCH requests onto specific controller handler methods.

How it Works: Shortcut for @RequestMapping(method = RequestMethod.PATCH). Performs partial modifications to an existing resource.

Usage in LibraryMS: Used on UserController to toggle active user status or change roles without re-submitting complete user payloads.

```
@PatchMapping("/{id}/status")
public ResponseEntity updateStatus(
        @PathVariable Long id,
        @RequestBody UpdateStatusRequest request) { ... }
```

@DeleteMapping

Package: org.springframework.web.bind.annotation.DeleteMapping

Purpose: Composed shortcut annotation that maps HTTP DELETE requests onto specific controller handler methods.

How it Works: Shortcut for @RequestMapping(method = RequestMethod.DELETE). Removes resources idempotently.

Usage in LibraryMS: Used to delete books, remove authors, and delete physical book copies.

```
@DeleteMapping("/{id}")
public ResponseEntity deleteBook(@PathVariable Long id) {
    bookService.deleteBook(id);
    return ResponseEntity.noContent().build();
}
```

@RequestBody

Package: org.springframework.web.bind.annotation.RequestBody

Purpose: Binds the incoming HTTP request body to a method parameter, deserializing it via HttpMessageConverter (Jackson).

How it Works: Reads the input stream, converts JSON into immutable Java records or DTOs, and enables validation triggers.

Usage in LibraryMS: Used on creation and update endpoints: CreateBookRequest, LoginRequest, UpdateMemberRequest.

```
@PostMapping
public ResponseEntity createBook(@Valid @RequestBody CreateBookRequest request) { ... }
```

@PathVariable

Package: org.springframework.web.bind.annotation.PathVariable

Purpose: Extracts dynamic URI template variables from the request path into method arguments.

How it Works: Spring parses URL placeholders like /{id} and binds them to the annotated method parameter with automatic type conversion.

Usage in LibraryMS: Used on all single-resource endpoints: GET /books/{id}, PUT /members/{id}, DELETE /copies/{id}.

```
@GetMapping("/{id}")
public ResponseEntity getBookById(@PathVariable Long id) { ... }
```

@RequestParam

Package: org.springframework.web.bind.annotation.RequestParam

Purpose: Binds query parameters from the HTTP request URL (?title=Spring&genre;=Tech) to method parameters.

How it Works: Extracts query string parameters, handles required/optional states, and provides default values.

Usage in LibraryMS: Used in list endpoints for searching, filtering, and criteria lookups (e.g., listBooks, listLoans, audit criteria).

```
@GetMapping
public ResponseEntity> listBooks(
        @RequestParam(required = false) String title,
        @RequestParam(required = false) String genre) { ... }
```

@DateTimeFormat

Package: org.springframework.format.annotation.DateTimeFormat

Purpose: Declares that a field or method parameter should be formatted as a date or time according to ISO or custom patterns.

How it Works: Configures DateTimeFormatter to parse incoming query string timestamps into Java 8+ java.time.Instant or LocalDate objects.

Usage in LibraryMS: Used in AuditController to parse 'from' and 'to' temporal audit query filters in ISO-8601 date-time format.

```
@GetMapping
public ResponseEntity> getAuditLogs(
        @RequestParam(required = false) @DateTimeFormat(iso = DateTimeFormat.ISO.DATE_TIME) Instant from,
        @RequestParam(required = false) @DateTimeFormat(iso = DateTimeFormat.ISO.DATE_TIME) Instant to) { ...
}
```

@PageableDefault

Package: org.springframework.data.web.PageableDefault

Purpose: Configures default pagination and sorting parameters for Spring Data Pageable controller arguments.

How it Works: Injects a default PageRequest (page size, initial page index, sort field and direction) when the client omits query parameters.

Usage in LibraryMS: Used across all paginated GET endpoints in BookController, MemberController, LoanController, and AuditController.

```
@GetMapping
public ResponseEntity> listLoans(
        @PageableDefault(size = 20, sort = "borrowedAt", direction = Sort.Direction.DESC) Pageable pageable) {
 ... }
```

@RestControllerAdvice

Package: org.springframework.web.bind.annotation.RestControllerAdvice

Purpose: Declares global, cross-cutting exception handlers and response modifiers across all @RestController classes.

How it Works: Combines @ControllerAdvice and @ResponseBody to intercept unhandled exceptions and format standard RFC-compliant JSON responses.

Usage in LibraryMS: Used on GlobalExceptionHandler to catch ApiException, validation errors, and bad credentials globally.

```
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity handleNotFound(...) { ... }
}
```

@ExceptionHandler

Package: org.springframework.web.bind.annotation.ExceptionHandler

Purpose: Designates a method as the designated handler for specific exception types thrown within controller invocations.

How it Works: Spring MVC catches matching exceptions thrown by controllers or services and routes execution to this method.

Usage in LibraryMS: Used in GlobalExceptionHandler to return specific HTTP status codes (404 for NotFound, 409 for Conflict, 400 for BadRequest).

```
@ExceptionHandler(ConflictException.class)
public ResponseEntity handleConflict(ConflictException ex) {
    return ResponseEntity.status(HttpStatus.CONFLICT).body(...);
}
```

3. Spring Security & Access Control

@EnableWebSecurity

Package: org.springframework.security.config.annotation.web.configuration.EnableWebSecurity

Purpose: Activates Spring Security's web security support and provides the Spring MVC integration.

How it Works: Instructs Spring to create the DelegatingFilterProxy and instantiate the security filter chain.

Usage in LibraryMS: Used on SecurityConfig to configure JWT authentication, CORS, CSRF, and stateless session rules.

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig { ... }
```

@EnableMethodSecurity

Package: org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity

Purpose: Enables Spring Security method-level authorization annotations, specifically @PreAuthorize and @PostAuthorize.

How it Works: Creates Spring AOP pointcuts on methods having security annotations and intercepts calls before execution to check roles.

Usage in LibraryMS: Used on SecurityConfig to enable role-based endpoint protection across controllers.

```
@Configuration
@EnableMethodSecurity
public class SecurityConfig { ... }
```

@PreAuthorize

Package: org.springframework.security.access.prepost.PreAuthorize

Purpose: Evaluates a SpEL (Spring Expression Language) security expression before permitting a method invocation.

How it Works: Evaluates the expression against the active SecurityContext Authentication principal. If false, throws AccessDeniedException (HTTP 403).

Usage in LibraryMS: Used to protect admin endpoints (hasRole('ROLE_ADMIN')) and ownership checks (@loanSecurity.isLoanMember(...)).

```
@PostMapping("/{id}/return")
@PreAuthorize("hasRole('ROLE_ADMIN')")
public ResponseEntity returnLoan(@PathVariable Long id) { ... }

@GetMapping("/{id}")
@PreAuthorize("hasRole('ROLE_ADMIN') or @loanSecurity.isLoanMember(authentication, #id)")
```

@AuthenticationPrincipal

Package: org.springframework.security.core.annotation.AuthenticationPrincipal

Purpose: Resolves the current authenticated user's principal object directly into a controller method argument.

How it Works: AuthenticationPrincipalArgumentResolver extracts SecurityContextHolder.getContext().getAuthentication().getPrincipal().

Usage in LibraryMS: Used in UserController (/users/me), MemberController (/members/me), and LoanController (/loans) to identify the logged-in reader.

```
@GetMapping("/me")
public ResponseEntity getMe(@AuthenticationPrincipal CurrentUserPrincipal principal) {
    return ResponseEntity.ok(userService.getUserById(principal.id()));
}
```

4. Spring Data JPA & Transaction Management

@Transactional

Package: org.springframework.transaction.annotation.Transactional

Purpose: Defines database transaction boundaries on methods or classes, managing commit, rollback, and isolation.

How it Works: Spring AOP wraps the method in a TransactionInterceptor. Successful execution triggers a commit; unchecked exceptions trigger an automatic rollback.

Usage in LibraryMS: Applied across all service methods: readOnly=true for queries, and mandatory/new propagation for auditing.

```
@Transactional
public LoanResponse createLoan(CreateLoanRequest request, CurrentUserPrincipal principal) { ... }

@Transactional(readOnly = true)
public BookResponse getBookById(Long id) { ... }
```

@EnableTransactionManagement

Package: org.springframework.transaction.annotation.EnableTransactionManagement

Purpose: Enables Spring's annotation-driven transaction management capability (processing @Transactional).

How it Works: Registers TransactionInterceptor and PlatformTransactionManager proxies. Specifying order=0 ensures transactions start before outer aspects.

Usage in LibraryMS: Used in AuditConfig (order = 0) so that @Transactional wraps @Audited aspect invocations.

```
@Configuration
@EnableTransactionManagement(order = 0)
public class AuditConfig { ... }
```

@Query

Package: org.springframework.data.jpa.repository.Query

Purpose: Declares custom JPQL or native SQL queries directly on Spring Data JPA repository methods.

How it Works: Spring Data executes the declared query instead of deriving it from method name conventions.

Usage in LibraryMS: Used in LoanRepository, BookRepository, and RefreshTokenRepository for specialized joins, detailed entity fetching, and partial unique checks.

```
@Query("SELECT l FROM Loan l JOIN FETCH l.copy c JOIN FETCH c.book JOIN FETCH l.member WHERE l.id = :id")
Optional findDetailedById(@Param("id") Long id);
```

@Modifying

Package: org.springframework.data.jpa.repository.Modifying

Purpose: Informs Spring Data JPA that the query annotated with @Query is an UPDATE or DELETE DML statement rather than a SELECT.

How it Works: Executes the query using executeUpdate() on the EntityManager and optionally flushes and clears the persistence context.

Usage in LibraryMS: Used in RefreshTokenRepository to bulk-revoke tokens on user deletion or password changes.

```
@Modifying
@Query("UPDATE RefreshToken r SET r.revokedAt = :now WHERE r.user.id = :userId AND r.revokedAt IS NULL")
void revokeAllForUser(@Param("userId") Long userId, @Param("now") Instant now);
```

@Param

Package: org.springframework.data.repository.query.Param

Purpose: Binds method parameters to named bind variables in JPQL/SQL queries (:id, :userId).

How it Works: Prevents SQL injection by ensuring parameters are securely bound as prepared statement variables.

Usage in LibraryMS: Used in all custom @Query methods in LoanRepository, UserRepository, and RefreshTokenRepository.

```
Optional findByEmail(@Param("email") String email);
```

@EntityGraph

Package: org.springframework.data.jpa.repository.EntityGraph

Purpose: Specifies an entity graph to eagerly load related associations in a single SQL JOIN, eliminating N+1 query problems.

How it Works: Instructs Hibernate to override fetch=LAZY for the specified attributePaths during this specific query execution.

Usage in LibraryMS: Used in UserRepository (attributePaths = "roles") to fetch user roles eagerly during JWT validation.

```
@EntityGraph(attributePaths = {"roles"})
Optional findWithRolesById(Long id);
```

5. Jakarta Persistence (JPA / Hibernate ORM)

@Entity

Package: jakarta.persistence.Entity

Purpose: Specifies that the class is an entity and its lifecycle is managed by the JPA persistence context.

How it Works: Hibernate registers the class as a persistent relational mapping table in the metamodel.

Usage in LibraryMS: Used on Book, Author, Member, Loan, BookCopy, User, Role, RefreshToken, and AuditLog.

```
@Entity
@Table(name = "books")
public class Book extends BaseEntity { ... }
```

@Table

Package: jakarta.persistence.Table

Purpose: Specifies the primary database table name and optional catalog/schema for an annotated entity.

How it Works: Maps the Java class name to the physical PostgreSQL table name (e.g. book_copies, audit_logs).

Usage in LibraryMS: Annotates all entity classes in com.libraryms.model to enforce lowercase plural SQL table names.

```
@Entity
@Table(name = "book_copies")
public class BookCopy extends BaseEntity { ... }
```

@Id

Package: jakarta.persistence.Id

Purpose: Specifies the primary key of an entity.

How it Works: Declares the field as the unique identifier used by Hibernate's first-level cache (Identity Map).

Usage in LibraryMS: Used in BaseEntity and Role to define 64-bit Long primary keys.

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
```

@GeneratedValue

Package: jakarta.persistence.GeneratedValue

Purpose: Configures the specification of generation strategies for the values of primary keys.

How it Works: GenerationType.IDENTITY utilizes PostgreSQL's GENERATED ALWAYS AS IDENTITY column to assign unique BIGINT IDs on INSERT.

Usage in LibraryMS: Paired with @Id on BaseEntity and Role to delegate sequence incrementing to PostgreSQL.

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
@Column(name = "id", nullable = false, updatable = false)
private Long id;
```

@Column

Package: jakarta.persistence.Column

Purpose: Configures mapping attributes of a persistent column (column name, nullable, length, unique, columnDefinition).

How it Works: Enforces Hibernate schema validation to match Flyway V1 migration constraints.

Usage in LibraryMS: Used on entity fields (e.g. length=200 for author names, nullable=false, columnDefinition='text' for bios).

```
@Column(name = "name", nullable = false, length = 200)
private String name;
```

@Version

Package: jakarta.persistence.Version

Purpose: Enables Optimistic Locking by automatically incrementing a version number column on each entity update.

How it Works: When updating, Hibernate executes: UPDATE ... SET version = version + 1 WHERE id = ? AND version = ?. If rows updated == 0, throws OptimisticLockException.

Usage in LibraryMS: Declared in BaseEntity so Book, BookCopy, Member, and Loan automatically have race-condition safety.

```
@Version
@Column(name = "version", nullable = false)
private Long version = 0L;
```

@Enumerated

Package: jakarta.persistence.Enumerated

Purpose: Specifies that an enum field should be stored in the database as either an ORDINAL (integer) or STRING (text).

How it Works: EnumType.STRING persists the literal name (e.g. 'AVAILABLE', 'ON_LOAN', 'LOST') ensuring safety against enum reordering.

Usage in LibraryMS: Used on CopyStatus in BookCopy, and RoleName in Role.

```
@Enumerated(EnumType.STRING)
@Column(name = "status", nullable = false, length = 16)
private CopyStatus status = CopyStatus.AVAILABLE;
```

@ManyToOne

Package: jakarta.persistence.ManyToOne

Purpose: Defines a single-valued association to another entity class that has many-to-one multiplicity.

How it Works: Configures foreign key relationships. Setting fetch = FetchType.LAZY ensures related entities are only loaded when explicitly referenced.

Usage in LibraryMS: Used in Loan (ManyToOne to BookCopy and Member) and BookCopy (ManyToOne to Book).

```
@ManyToOne(fetch = FetchType.LAZY, optional = false)
@JoinColumn(name = "copy_id", nullable = false)
private BookCopy copy;
```

@OneToOne

Package: jakarta.persistence.OneToOne

Purpose: Defines a single-valued association to another entity that has one-to-one multiplicity.

How it Works: Maps a unique 1:1 foreign key relationship between two distinct tables.

Usage in LibraryMS: Used in User to map the optional 1:1 linkage to Member.

```
@OneToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "member_id", unique = true)
private Member member;
```

@ManyToMany

Package: jakarta.persistence.ManyToMany

Purpose: Defines a many-to-many relationship between two entities mapped through an intermediate join table.

How it Works: Hibernate manages the join table rows automatically when entities are associated or disassociated.

Usage in LibraryMS: Used in Book (books have multiple authors; authors have multiple books) and User (users have multiple roles).

```
@ManyToMany
@JoinTable(name = "book_authors",
    joinColumns = @JoinColumn(name = "book_id"),
    inverseJoinColumns = @JoinColumn(name = "author_id"))
private Set authors = new HashSet<>();
```

@JoinColumn

Package: jakarta.persistence.JoinColumn

Purpose: Specifies a column for joining an entity association or element collection (the physical foreign key column).

How it Works: Declares the foreign key column name in the database table and defines nullable/updatable properties.

Usage in LibraryMS: Used across all relational mappings: in Loan for copy_id and member_id, and in BookCopy for book_id.

```
@ManyToOne(fetch = FetchType.LAZY, optional = false)
@JoinColumn(name = "member_id", nullable = false)
private Member member;
```

@JoinTable

Package: jakarta.persistence.JoinTable

Purpose: Specifies the mapping of associations involving an intermediate join table.

How it Works: Defines the join table name, the owning foreign key (joinColumns), and the inverse foreign key (inverseJoinColumns).

Usage in LibraryMS: Used in Book for book_authors and User for user_roles join tables.

```
@ManyToMany
@JoinTable(name = "user_roles",
    joinColumns = @JoinColumn(name = "user_id"),
    inverseJoinColumns = @JoinColumn(name = "role_id"))
private Set roles = new HashSet<>();
```

@ElementCollection

Package: jakarta.persistence.ElementCollection

Purpose: Specifies a collection of basic types or embeddable classes that do not have their own primary identity.

How it Works: Instructs Hibernate that the target elements are basic values (e.g. Set) stored in a auxiliary linked table.

Usage in LibraryMS: Used in Book to map genres (Set genres) without needing a dedicated Genre entity.

```
@ElementCollection
@CollectionTable(name = "book_genres", joinColumns = @JoinColumn(name = "book_id"))
@Column(name = "genre", nullable = false, length = 50)
private Set genres = new HashSet<>();
```

@CollectionTable

Package: jakarta.persistence.CollectionTable

Purpose: Specifies the table that is used for the mapping of collections of basic or embeddable types.

How it Works: Defines the foreign key column linking the child value rows back to the parent entity's primary key.

Usage in LibraryMS: Used in Book to name the book_genres table and link it via book_id foreign key.

```
@CollectionTable(name = "book_genres", joinColumns = @JoinColumn(name = "book_id"))
private Set genres = new HashSet<>();
```

@MappedSuperclass

Package: jakarta.persistence.MappedSuperclass

Purpose: Designates a class whose mapping information is inherited by its subclasses. No dedicated table is generated for it.

How it Works: All entity tables inheriting from the class include its declared columns (id, version, createdAt, updatedAt).

Usage in LibraryMS: Used on BaseEntity, providing unified ID, optimistic locking, and audit timestamps across all aggregates.

```
@MappedSuperclass
public abstract class BaseEntity {
    @Id @GeneratedValue ... private Long id;
    @Version ... private Long version;
    private Instant createdAt;
    private Instant updatedAt;
}
```

@PrePersist

Package: jakarta.persistence.PrePersist

Purpose: Specifies a callback method that is executed before the entity is persisted (INSERT) into the database.

How it Works: Hibernate invokes this method during persistence context flush before executing the INSERT SQL statement.

Usage in LibraryMS: Used in BaseEntity to initialize createdAt and updatedAt timestamps automatically.

```
@PrePersist
protected void onCreate() {
    Instant now = Instant.now();
    this.createdAt = now;
    this.updatedAt = now;
}
```

@PreUpdate

Package: jakarta.persistence.PreUpdate

Purpose: Specifies a callback method that is executed before the entity is updated (UPDATE) in the database.

How it Works: Hibernate invokes this method during persistence context flush before executing the UPDATE SQL statement.

Usage in LibraryMS: Used in BaseEntity to refresh the updatedAt timestamp whenever any entity property changes.

```
@PreUpdate
protected void onUpdate() {
    this.updatedAt = Instant.now();
}
```

6. Jakarta Bean Validation & Custom Constraints

@Valid

Package: jakarta.validation.Valid

Purpose: Triggers recursive cascade validation on an object parameter or nested property.

How it Works: In controller methods, instructs Spring to validate incoming @RequestBody payloads against constraint annotations.

Usage in LibraryMS: Used across all controller POST and PUT methods to enforce DTO contracts before service execution.

```
@PostMapping
public ResponseEntity createBook(@Valid @RequestBody CreateBookRequest request) { ... }
```

@Validated

Package: org.springframework.validation.annotation.Validated

Purpose: Spring-specific variant of @Valid enabling validation on Spring beans, method parameters, and group validation.

How it Works: Wraps the class in a MethodValidationPostProcessor proxy to validate path variables and request parameters.

Usage in LibraryMS: Annotates all REST controllers (e.g. @Validated public class LoanController) to validate path IDs and filters.

```
@RestController
@RequestMapping("/api/v1/loans")
@Validated
public class LoanController { ... }
```

@NotNull

Package: jakarta.validation.constraints.NotNull

Purpose: Restricts that the annotated property value cannot be null.

How it Works: Validates that a field is non-null. Unlike @NotBlank, allows empty strings or empty collections.

Usage in LibraryMS: Used on dates, IDs, and numeric properties: publishedDate, authorIds, and membership dates.

```
@NotNull(message = "Published date is required")
LocalDate publishedDate
```

@NotBlank

Package: jakarta.validation.constraints.NotBlank

Purpose: Restricts that the annotated character sequence cannot be null and must have at least one non-whitespace character.

How it Works: Trims whitespace and checks length > 0. Stricter than @NotNull or @NotEmpty.

Usage in LibraryMS: Used on required text fields: title, isbn, username, email, and password.

```
@NotBlank(message = "Title is required")
String title
```

@NotEmpty

Package: jakarta.validation.constraints.NotEmpty

Purpose: Restricts that the annotated collection, map, array, or string cannot be null and must have size > 0.

How it Works: Checks that collections or arrays have at least one element.

Usage in LibraryMS: Used on book author sets to ensure a book is associated with at least one author ID upon creation.

```
@NotEmpty(message = "At least one author ID is required")
Set authorIds
```

@Size

Package: jakarta.validation.constraints.Size

Purpose: Constrains the size of a string, collection, map, or array between specified min and max bounds.

How it Works: Validates length of characters. Exceeding max produces a MethodArgumentNotValidException.

Usage in LibraryMS: Used on title (max 500), author name (max 200), and username (min 3, max 50).

```
@Size(max = 500, message = "Title cannot exceed 500 characters")
String title
```

@Email

Package: jakarta.validation.constraints.Email

Purpose: Verifies that a string conforms to valid RFC email address syntax.

How it Works: Applies a standard regular expression pattern matching email formats.

Usage in LibraryMS: Used in RegisterRequest and CreateMemberRequest to validate patron and user email addresses.

```
@NotBlank(message = "Email is required")
@Email(message = "Invalid email format")
String email
```

@Past

Package: jakarta.validation.constraints.Past

Purpose: Ensures that an annotated Date, LocalDate, or Instant is strictly in the past relative to the current clock time.

How it Works: Compares value against the system clock. Rejects future or today dates.

Usage in LibraryMS: Used in CreateAuthorRequest and UpdateAuthorRequest to ensure author birth dates are strictly historical.

```
@Past(message = "Birth date must be in the past")
LocalDate birthDate
```

@ValidIsbn

Package: com.libraryms.config.ValidIsbn

Purpose: Custom domain Bean Validation constraint validating ISBN-10 and ISBN-13 checksum algorithms.

How it Works: Declared with @Constraint(validatedBy = IsbnValidator.class); delegates validation logic to the IsbnValidator component.

Usage in LibraryMS: Used in CreateBookRequest and UpdateBookRequest to validate ISBN formats prior to database insertion.

```
@ValidIsbn
private String isbn;

// Definition:
@Target({ElementType.FIELD, ElementType.PARAMETER})
@Retention(RetentionPolicy.RUNTIME)
@Constraint(validatedBy = IsbnValidator.class)
public @interface ValidIsbn { ... }
```

@Constraint

Package: jakarta.validation.Constraint

Purpose: Meta-annotation applied to custom constraint annotations to declare the validator implementation class.

How it Works: Jakarta Bean Validation engine discovers the validator class and instantiates it to process values.

Usage in LibraryMS: Used to wire ValidIsbn to its IsbnValidator implementation class.

```
@Constraint(validatedBy = IsbnValidator.class)
public @interface ValidIsbn {
    String message() default "Invalid ISBN-10 or ISBN-13 format";
    Class[] groups() default {};
    Class[] payload() default {};
}
```

7. Spring AOP & Aspect-Oriented Programming

@Aspect

Package: org.aspectj.lang.annotation.Aspect

Purpose: Declares a class as an Aspect in AspectJ / Spring AOP, encapsulating cross-cutting concerns.

How it Works: Spring detects the bean as an interceptor and proxies target beans matching pointcut expressions.

Usage in LibraryMS: Used on AuditAspect to intercept service methods and record audit history without polluting domain code.

```
@Aspect
@Component
@Order(100)
public class AuditAspect { ... }
```

@Around

Package: org.aspectj.lang.annotation.Around

Purpose: Defines an advice that surrounds a join point method execution, giving complete control over proceeding.

How it Works: ProceedingJoinPoint.proceed() executes the wrapped method. Code before runs before; code after runs post-execution.

Usage in LibraryMS: Used in AuditAspect with @Around("@annotation(audited)") to intercept methods decorated with @Audited.

```
@Around("@annotation(audited)")
public Object auditMethod(ProceedingJoinPoint pjp, Audited audited) throws Throwable {
    Object result = pjp.proceed();
    auditService.logEvent(...);
    return result;
}
```

@Audited

Package: com.libraryms.config.Audited

Purpose: Custom declarative annotation marking service methods that must automatically produce audit log records.

How it Works: Custom retention RUNTIME annotation detected by AuditAspect's pointcut.

Usage in LibraryMS: Placed on service methods throughout BookService, LoanService, MemberService, and UserService.

```
@Transactional
@Audited(entityType = "BOOK", operation = "CREATE")
public BookResponse createBook(CreateBookRequest request) { ... }
```

@EnableAspectJAutoProxy

Package: org.springframework.context.annotation.EnableAspectJAutoProxy

Purpose: Enables Spring's support for handling components marked with AspectJ's @Aspect annotation.

How it Works: Registers an AnnotationAwareAspectJAutoProxyCreator in the bean definition registry.

Usage in LibraryMS: Used in AuditConfig to activate AOP proxy interception across the application.

```
@Configuration
@EnableAspectJAutoProxy
public class AuditConfig { ... }
```

8. MapStruct Object Mapping

@Mapper

Package: org.mapstruct.Mapper

Purpose: Marks an interface as a MapStruct mapping contract. Generates type-safe DTO-to-entity code at compile time.

How it Works: Annotation processor generates an implementation class (e.g. BookMapperImpl) with direct getters/setters.

Usage in LibraryMS: Used on BookMapper, AuthorMapper, LoanMapper, MemberMapper, and UserMapper.

```
@Mapper(componentModel = "spring", unmappedTargetPolicy = ReportingPolicy.ERROR)
public interface BookMapper { ... }
```

@Mapping

Package: org.mapstruct.Mapping

Purpose: Configures the mapping of a specific target field from a source property, expression, or custom method qualifier.

How it Works: Generates explicit assignment statements in the generated implementation class.

Usage in LibraryMS: Used in BookMapper, LoanMapper, and UserMapper to map nested relationships (e.g. member.id -> memberId).

```
@Mapping(target = "memberId", source = "member.id")
@Mapping(target = "roles", source = "roles", qualifiedByName = "mapRoleNames")
UserResponse toResponse(User user);
```

@MappingTarget

Package: org.mapstruct.MappingTarget

Purpose: Designates a parameter of a mapping method as the target object to be updated in-place.

How it Works: Instead of instantiating a new object via 'new', MapStruct writes values directly into the passed instance.

Usage in LibraryMS: Used in AuthorMapper and BookMapper for update operations, modifying existing JPA managed entities.

```
@Mapping(target = "id", ignore = true)
@Mapping(target = "version", ignore = true)
void updateEntity(UpdateAuthorRequest request, @MappingTarget Author author);
```

@Named

Package: org.mapstruct.Named

Purpose: Declares a qualifier name on a mapping method that can be referenced by @Mapping(qualifiedByName = ...).

How it Works: Allows MapStruct to distinguish between multiple mapping methods with the same signature.

Usage in LibraryMS: Used in UserMapper to specify custom role conversion logic (Set -> Set).

```
@Named("mapRoleNames")
default Set mapRoleNames(Set roles) {
    return roles.stream().map(r -> r.getName().name()).collect(Collectors.toSet());
}
```

9. Lombok Boilerplate Reduction

@Getter

Package: lombok.Getter

Purpose: Generates default getter methods for all class fields or specific annotated fields at compile time.

How it Works: Lombok's annotation processor modifies the AST during compilation, injecting public getX() methods into bytecode.

Usage in LibraryMS: Used across all JPA model entities: Book, Author, Member, Loan, User, BookCopy, RefreshToken, and AuditLog.

```
@Entity
@Getter
@Setter
@NoArgsConstructor
public class Author extends BaseEntity {
    private String name;
    private String bio;
}
```

@Setter

Package: lombok.Setter

Purpose: Generates default setter methods for mutable non-final fields at compile time.

How it Works: Lombok injects public setX(...) methods into the compiled bytecode, eliminating repetitive setter code.

Usage in LibraryMS: Used on entities like Book, Author, Member, and Loan for property mutations and MapStruct integration.

```
@Entity
@Getter
@Setter
public class Book extends BaseEntity {
    private String title;
    private String isbn;
}
```

@NoArgsConstructor

Package: lombok.NoArgsConstructor

Purpose: Generates a default no-argument constructor.

How it Works: Injects a public no-arg constructor required by the JPA specification (Hibernate instantiation through reflection).

Usage in LibraryMS: Used across all JPA model entities in com.libraryms.model to fulfill JPA requirements.

```
@Entity
@Table(name = "members")
@Getter
@Setter
@NoArgsConstructor
public class Member extends BaseEntity { ... }
```